Algorithms 41–51

Flowcharts • Pseudocode • Python Code • Sample Outputs

A practical programming workbook covering nested-list matrix analysis and tuple-based audits.

How to use this PDF

Each topic includes an aim, step-by-step algorithm, a text flowchart, pseudocode, runnable Python example, and representative output. The sample data and thresholds are illustrative; adjust them to your assignment requirements.

Note on numbering: The supplied list labels topics 41–50 but contains ten topic descriptions after 41. The final route-distance topic is labeled 51 here to keep the sequence continuous.

41. Traffic Flow and Congestion Analysis

Aim: Process vehicle counts for multiple roads and time intervals, find peak periods, classify congestion, and flag roads above capacity.

Algorithm

1. Read road names, capacity per interval, and vehicle counts for each time interval.

2. For each road, calculate its maximum count and the interval where it occurs.

3. Classify each count: Low below 50% of capacity, Moderate from 50% to below 80%, High from 80% to capacity, and Over capacity above capacity.

4. Report each road's peak interval and flag any road whose maximum exceeds capacity.

Flowchart

```
START
  |
Read roads, counts, capacities
  |
For each road, find peak count/time
  |
Compare count with capacity thresholds
  |
Print congestion level and violations
  |
 END
```

Pseudocode

```
INPUT roads, counts, capacity
FOR each road r:
    peak = maximum(counts[r])
    peak_time = time of peak
    FOR each interval count c in counts[r]:
        IF c < 0.50 * capacity[r]: level = Low
        ELSE IF c < 0.80 * capacity[r]: level = Moderate
        ELSE IF c <= capacity[r]: level = High
        ELSE: level = Over capacity
        DISPLAY interval, c, level
    IF peak > capacity[r]: FLAG road
    DISPLAY peak, peak_time
```

Python Code

```
roads = {
    "North Road": [40, 75, 110, 90],
    "Market Road": [60, 95, 120, 130]
}
capacity = {"North Road": 100, "Market Road": 120}
times = ["08:00", "09:00", "10:00", "11:00"]

for road, counts in roads.items():
    peak = max(counts)
    peak_time = times[counts.index(peak)]
    print(f"{road}: peak {peak} vehicles at {peak_time}")
    for time, count in zip(times, counts):
        ratio = count / capacity[road]
        if ratio < 0.50:
            level = "Low"
        elif ratio < 0.80:
            level = "Moderate"
        elif ratio <= 1.00:
            level = "High"
        else:
            level = "Over capacity"
        print(f"  {time}: {count} - {level}")
    if peak > capacity[road]:
        print("  ALERT: capacity exceeded")
```

Sample Output

```
North Road: peak 110 vehicles at 10:00
  08:00: 40 - Low
  09:00: 75 - Moderate
  10:00: 110 - Over capacity
  11:00: 90 - High
  ALERT: capacity exceeded
Market Road: peak 130 vehicles at 11:00
  08:00: 60 - Moderate
  09:00: 95 - High
```

```
10:00: 120 - High
11:00: 130 - Over capacity
ALERT: capacity exceeded
```

Note: Thresholds are illustrative and can be adjusted to local traffic standards.

42. Nested List Classroom Occupancy Analysis

Aim: Allocate students to classroom seats while preserving reserved seats, detect duplicate assignments, and calculate vacant capacity.

Algorithm

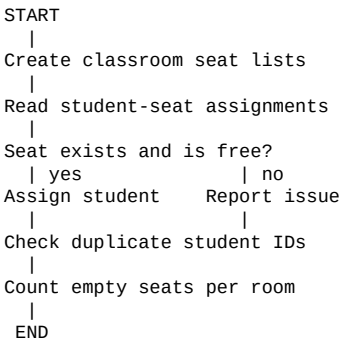
1. Represent each classroom as a nested list of seat labels; use None for an empty seat and 'RESERVED' for a reserved seat.

2. Process student-seat assignments, checking that the seat exists and is not reserved or occupied.

3. Track assigned student IDs and report duplicate assignments.

4. Count empty seats and report vacant capacity for each classroom.

Flowchart



Pseudocode

```
FOR each assignment (student, room, seat):
    IF student already assigned: report duplicate
    ELSE IF seat is reserved or occupied: report unavailable
    ELSE place student and record assignment
FOR each room:
    vacant = number of seats equal to None
    DISPLAY vacant and total capacity
```

Python Code

```
classrooms = {
    "A": [None, "RESERVED", None, None],
    "B": [None, None, None]
}
assignments = [("S01", "A", 0), ("S02", "A", 2),
               ("S01", "B", 1), ("S03", "A", 1)]
assigned_students = set()

for student, room, seat in assignments:
    if student in assigned_students:
        print(f"Duplicate assignment: {student}")
        continue
    if room not in classrooms or not 0 <= seat < len(classrooms[room]):
        print(f"Invalid seat: {room}[{seat}]")
        continue
    if classrooms[room][seat] is not None:
        print(f"Seat unavailable: {room}[{seat}]")
        continue
    classrooms[room][seat] = student
    assigned_students.add(student)
    print(f"Assigned {student} to {room}[{seat}]")

for room, seats in classrooms.items():
    vacant = seats.count(None)
    print(f"Room {room}: {vacant} vacant of {len(seats)} seats")
```

Sample Output

```
Assigned S01 to A[0]
Assigned S02 to A[2]
Duplicate assignment: S01
Seat unavailable: A[1]
Room A: 1 vacant of 4 seats
Room B: 3 vacant of 3 seats
```

Note: Seat positions use zero-based indexing in Python.

43. Student Marks Matrix Analysis

Aim: Calculate student totals and averages, subject averages, top performers, and subjects with low average marks.

Algorithm

1. Store marks as a matrix: each row is a student and each column is a subject.

2. Calculate each student's total and average.

3. Calculate each subject's average down the columns.

4. Find the highest-scoring student and report subjects whose average is below the chosen threshold.

Flowchart

```
START
  |
Read marks matrix and labels
  |
Calculate row totals/averages
  |
Calculate column averages
  |
Find top student and low-average subjects
  |
Display results
  |
 END
```

Pseudocode

```
FOR each student row:
    total = sum(row)
    average = total / number_of_subjects
FIND student with maximum total
FOR each subject column:
    average = sum(column) / number_of_students
    IF average < threshold: mark as low-performing
DISPLAY all statistics
```

Python Code

```
students = ["Asha", "Bala", "Charan"]
subjects = ["Math", "Science", "English"]
marks = [[88, 76, 91], [72, 68, 75], [95, 92, 89]]
low_threshold = 70

totals = [sum(row) for row in marks]
for name, row, total in zip(students, marks, totals):
    print(f"{name}: total={total}, average={total / len(subjects):.1f}")

top = totals.index(max(totals))
print(f"Top performer: {students[top]} ({totals[top]} marks)")

for j, subject in enumerate(subjects):
    avg = sum(row[j] for row in marks) / len(students)
    print(f"{subject} average: {avg:.1f}")
    if avg < low_threshold:
        print(f"  LOW AVERAGE: {subject}")
```

Sample Output

```
Asha: total=255, average=85.0
Bala: total=215, average=71.7
Charan: total=276, average=92.0
Top performer: Charan (276 marks)
Math average: 85.0
Science average: 78.7
English average: 85.0
```

Note: This example flags subjects below 70; none are below the threshold.

44. Multi-Branch Sales Matrix Comparison

Aim: Store product sales by branch and day, calculate product totals, compare branches, and identify each branch's best product.

Algorithm

1. Store sales in a nested list where each branch contains a list of product sales aggregated across days.

2. Sum each product's sales across branches.

3. Calculate each branch's total sales.

4. Within each branch, identify the product with the largest sales value.

Flowchart

```
START
  |
Read branch/product sales matrix
  |
Sum each product across branches
  |
Sum each branch
  |
Find best product per branch
  |
Display comparisons
  |
 END
```

Pseudocode

```
FOR each product column:
    product_total = sum sales from all branches
FOR each branch row:
    branch_total = sum(row)
    best_product = product at maximum value in row
DISPLAY product totals, branch totals, best products
```

Python Code

```
branches = ["East", "West", "South"]
products = ["Tea", "Coffee", "Juice"]
# Each row is a branch; columns are product totals for the week.
sales = [[120, 150, 90], [100, 180, 110], [140, 130, 160]]

for i, branch in enumerate(branches):
    total = sum(sales[i])
    best = sales[i].index(max(sales[i]))
    print(f"{branch}: total={total}, best product={products[best]}")

for j, product in enumerate(products):
    total = sum(row[j] for row in sales)
    print(f"{product} total sales: {total}")

best_branch = max(range(len(branches)), key=lambda i: sum(sales[i]))
print(f"Best branch: {branches[best_branch]}")
```

Sample Output

```
East: total=360, best product=Coffee
West: total=390, best product=Coffee
South: total=430, best product=Juice
Tea total sales: 360
Coffee total sales: 460
Juice total sales: 360
Best branch: South
```

Note: For a day-by-day matrix, first sum each product's daily values within each branch.

45. Hospital Ward Occupancy Monitoring

Aim: Calculate ward utilization by day, detect capacity violations, and identify peak-demand periods.

Algorithm

1. Store occupied bed counts by ward and day in a nested list.

2. Keep each ward's bed capacity in a separate list or dictionary.

3. Calculate utilization percentage as occupied beds divided by capacity times 100.

4. Flag counts above capacity and identify the ward/day with the highest occupancy.

Flowchart

```
START
  |
Read occupancy matrix and capacities
  |
Calculate utilization for each ward/day
  |
Occupancy > capacity?
  | yes -> Flag violation
  |
Find maximum occupancy period
  |
Display report
  |
 END
```

Pseudocode

```
FOR each ward i and day j:
    utilization = occupancy[i][j] / capacity[i] * 100
    DISPLAY utilization
    IF occupancy[i][j] > capacity[i]: FLAG violation
FIND maximum value in occupancy matrix
DISPLAY ward and day of peak
```

Python Code

```
wards = ["Medical", "Surgical", "Maternity"]
days = ["Mon", "Tue", "Wed"]
capacity = [30, 25, 20]
occupied = [[24, 29, 32], [20, 22, 23], [15, 18, 17]]

peak = (-1, None, None)
for i, ward in enumerate(wards):
    for j, day in enumerate(days):
        count = occupied[i][j]
        utilization = count / capacity[i] * 100
        print(f"{ward} {day}: {count}/{capacity[i]} ({utilization:.1f}%)")
        if count > capacity[i]:
            print("  CAPACITY VIOLATION")
        if count > peak[0]:
            peak = (count, ward, day)

print(f"Peak demand: {peak[0]} beds, {peak[1]} ward on {peak[2]}")
```

Sample Output

```
Medical Mon: 24/30 (80.0%)
Medical Tue: 29/30 (96.7%)
Medical Wed: 32/30 (106.7%)
  CAPACITY VIOLATION
Surgical Mon: 20/25 (80.0%)
Surgical Tue: 22/25 (88.0%)
Surgical Wed: 23/25 (92.0%)
Maternity Mon: 15/20 (75.0%)
Maternity Tue: 18/20 (90.0%)
Maternity Wed: 17/20 (85.0%)
Peak demand: 32 beds, Medical ward on Wed
```

Note: Utilization can exceed 100% in the data to reveal a capacity breach.

46. Weekly Electricity Consumption Matrix

Aim: Calculate daily and appliance totals for seven days and flag consumption above specified limits.

Algorithm

1. Represent each appliance as a row containing seven daily readings in kWh.

2. Sum each row to get appliance-wise weekly consumption.

3. Sum each column to get total consumption for each day.

4. Flag any daily total or appliance total that exceeds its limit.

Flowchart

```
START
  |
Read 7-day appliance matrix and limits
  |
Calculate appliance row totals
  |
Calculate daily column totals
  |
Compare totals with limits
  |
Print totals and alerts
  |
 END
```

Pseudocode

```
FOR each appliance row:
    weekly_total = sum(row)
    IF weekly_total > appliance_limit: FLAG
FOR each day column:
    daily_total = sum(all appliance readings for day)
    IF daily_total > daily_limit: FLAG
DISPLAY totals and alerts
```

Python Code

```
appliances = ["AC", "Fridge", "Washing machine"]
# kWh readings from Monday to Sunday
usage = [
    [5, 6, 5, 7, 8, 6, 5],
    [2, 2, 2, 2, 2, 2, 2],
    [0, 0, 1, 0, 0, 2, 0]
]
appliance_limits = [40, 20, 10]
daily_limit = 9

for name, row, limit in zip(appliances, usage, appliance_limits):
    total = sum(row)
    print(f"{name}: weekly={total} kWh")
    if total > limit:
        print("  ALERT: appliance limit exceeded")

for day in range(7):
    total = sum(row[day] for row in usage)
    print(f"Day {day + 1}: {total} kWh")
    if total > daily_limit:
        print("  ALERT: daily limit exceeded")
```

Sample Output

```
AC: weekly=42 kWh
  ALERT: appliance limit exceeded
Fridge: weekly=14 kWh
Washing machine: weekly=3 kWh
Day 1: 7 kWh
Day 2: 8 kWh
Day 3: 8 kWh
Day 4: 9 kWh
Day 5: 10 kWh
  ALERT: daily limit exceeded
Day 6: 10 kWh
  ALERT: daily limit exceeded
Day 7: 7 kWh
```

Note: Readings are illustrative. Change limits to match the household or facility policy.

47. Inventory Matrix Reconciliation

Aim: Compare warehouse stock with target quantities, identify shortages and surpluses, and calculate transfers.

Algorithm

1. Store product quantities by warehouse in a matrix.

2. Compare each warehouse's quantity with its target for each product.

3. Compute shortage as target minus stock when positive, and surplus as stock minus target when positive.

4. Match surplus quantities to shortages for the same product and report suggested transfers.

Flowchart

```
START
  |
Read stock matrix and target matrix
  |
For each product/warehouse, compute surplus/shortage
  |
Match surplus to shortage for same product
  |
List suggested transfers
  |
 END
```

Pseudocode

```
FOR each product:
    identify warehouses with stock > target (surplus)
    identify warehouses with stock < target (shortage)
    FOR each shortage warehouse:
        transfer min(remaining surplus, shortage)
        update remaining surplus and shortage
DISPLAY transfers and any unmet shortage
```

Python Code

```
warehouses = ["A", "B", "C"]
products = ["P1", "P2"]
stock = [[50, 20], [10, 40], [30, 10]]
target = [[30, 25], [25, 25], [35, 20]]

for p, product in enumerate(products):
    surplus = []
    shortage = []
    for w, warehouse in enumerate(warehouses):
        diff = stock[w][p] - target[w][p]
        if diff > 0:
            surplus.append([warehouse, diff])
        elif diff < 0:
            shortage.append([warehouse, -diff])
    for need in shortage:
        for extra in surplus:
            amount = min(need[1], extra[1])
            if amount:
                print(f"{product}: transfer {amount} from {extra[0]} to {need[0]}")
                need[1] -= amount
                extra[1] -= amount
            if need[1] == 0:
                break
        if need[1] > 0:
            print(f"{product}: unmet shortage at {need[0]} = {need[1]}")
```

Sample Output

```
P1: transfer 15 from A to B
P1: transfer 5 from A to C
P2: transfer 5 from B to A
P2: transfer 10 from B to C
```

Note: Transfers are matched product-by-product; transportation costs and transfer restrictions are not modeled.

48. Sports Tournament Ranking System

Aim: Calculate team points and tie-break values from match results, then rank teams using a defined policy.

Algorithm

1. Initialize points and tie-break values for every team.

2. For each match, award 3 points for a win, 1 each for a draw, and 0 for a loss.

3. Accumulate goal difference (goals scored minus goals conceded) as the tie-break value.

4. Sort teams by points descending, then goal difference descending, then team name alphabetically.

Flowchart

```
START
  |
Initialize team statistics
  |
Process each match result
  |
Update points and goal difference
  |
Sort by points, goal difference, name
  |
Display ranking table
  |
 END
```

Pseudocode

```
FOR each match (teamA, teamB, goalsA, goalsB):
    update goals-for and goals-against
    IF goalsA > goalsB: A += 3 points
    ELSE IF goalsA < goalsB: B += 3 points
    ELSE: A += 1, B += 1
SORT teams by points DESC, goal difference DESC, name ASC
DISPLAY ranking
```

Python Code

```
teams = ["Falcons", "Tigers", "Eagles"]
matches = [
    ("Falcons", "Tigers", 2, 1),
    ("Eagles", "Falcons", 1, 1),
    ("Tigers", "Eagles", 3, 0)
]
stats = {t: {"points": 0, "gf": 0, "ga": 0} for t in teams}

for a, b, ga, gb in matches:
    stats[a]["gf"] += ga; stats[a]["ga"] += gb
    stats[b]["gf"] += gb; stats[b]["ga"] += ga
    if ga > gb:
        stats[a]["points"] += 3
    elif gb > ga:
        stats[b]["points"] += 3
    else:
        stats[a]["points"] += 1
        stats[b]["points"] += 1

ranking = sorted(teams, key=lambda t: (
    -stats[t]["points"],
    -(stats[t]["gf"] - stats[t]["ga"]),
    t
))
for rank, team in enumerate(ranking, 1):
    s = stats[team]
    gd = s["gf"] - s["ga"]
    print(f"{rank}. {team}: {s['points']} pts, GD {gd}")
```

Sample Output

```
1. Tigers: 3 pts, GD 0
2. Falcons: 2 pts, GD 1
3. Eagles: 1 pts, GD -1
```

Note: This policy uses 3 points for a win, 1 for a draw, and goal difference as the first tie-break.

49. Tuple-Based Employee Shift Conflict Detection

Aim: Represent shifts as tuples, detect overlapping time intervals, and report conflicting employee schedules.

Algorithm

1. Store each shift as a tuple: (employee, start_time, end_time), using minutes from midnight.

2. Compare pairs of shifts belonging to the same employee.

3. Two intervals overlap when start1 < end2 and start2 < end1.

4. Report each overlapping pair; shifts that only touch at an endpoint are not considered overlapping.

Flowchart

```
START
  |
Read shift tuples
  |
Compare each pair of shifts
  |
Same employee and intervals overlap?
  | yes -> Report conflict
  |
Continue until all pairs checked
  |
 END
```

Pseudocode

```
FOR i from 0 to number_of_shifts - 1:
    FOR j from i + 1 to number_of_shifts - 1:
        IF employee[i] == employee[j] AND
           start[i] < end[j] AND start[j] < end[i]:
            REPORT conflict between shifts i and j
```

Python Code

```
# Times are minutes after midnight: 09:00 = 540
shifts = [
    ("Mina", 540, 720),   # 09:00-12:00
    ("Mina", 690, 810),   # 11:30-13:30
    ("Ravi", 600, 720),   # 10:00-12:00
    ("Mina", 810, 900)    # 13:30-15:00
]

found = False
for i in range(len(shifts)):
    for j in range(i + 1, len(shifts)):
        e1, s1, end1 = shifts[i]
        e2, s2, end2 = shifts[j]
        if e1 == e2 and s1 < end2 and s2 < end1:
            print(f"Conflict for {e1}: shift {i + 1} overlaps shift {j + 1}")
            found = True
if not found:
    print("No conflicts found")
```

Sample Output

```
Conflict for Mina: shift 1 overlaps shift 2
```

Note: The end time is treated as exclusive, so 13:30-15:00 does not conflict with a shift ending at 13:30.

50. Tuple-Based Product Catalogue Audit

Aim: Validate product tuples, calculate inventory value, and identify products that need replenishment.

Algorithm

1. Store each product as a tuple: (product_id, price, quantity).

2. Validate that IDs are non-empty, price is non-negative, and quantity is a non-negative integer.

3. For valid records, calculate inventory value as price times quantity.

4. Flag products whose quantity is below the reorder threshold.

Flowchart

```
START
  |
Read product tuples and reorder level
  |
Validate each record
  |
Valid?
  | no -> Report invalid record
  | yes
Calculate price x quantity
  |
Quantity below reorder level?
  | yes -> Flag replenishment
  |
Display audit report
  |
 END
```

Pseudocode

```
FOR each (id, price, quantity):
    IF id empty OR price < 0 OR quantity is not integer OR quantity < 0:
        REPORT invalid record
    ELSE:
        value = price * quantity
        ADD value to inventory total
        IF quantity < reorder_level: FLAG replenishment
DISPLAY inventory total
```

Python Code

```
products = [
    ("P100", 25.50, 12),
    ("P200", 99.00, 3),
    ("", 10.00, 5),
    ("P400", 5.00, -2)
]
reorder_level = 5
inventory_total = 0

for product_id, price, quantity in products:
    if not product_id or price < 0 or not isinstance(quantity, int) or quantity < 0:
        print(f"Invalid record: {(product_id, price, quantity)}")
        continue
    value = price * quantity
    inventory_total += value
    print(f"{product_id}: value={value:.2f}")
    if quantity < reorder_level:
        print(f"  REORDER: stock is {quantity}")

print(f"Valid inventory value: {inventory_total:.2f}")
```

Sample Output

```
P100: value=306.00
P200: value=297.00
  REORDER: stock is 3
Invalid record: ('', 10.0, 5)
Invalid record: ('P400', 5.0, -2)
Valid inventory value: 603.00
```

Note: The inventory total excludes invalid records.

51. Tuple-Based Route Distance Analysis

Aim: Calculate distances between successive coordinate points using the Euclidean formula and identify the longest segment.

Algorithm

1. Store each route point as a tuple (x, y).

2. For every pair of successive points, calculate distance as sqrt((x2-x1)^2 + (y2-y1)^2).

3. Store each segment distance and compare it with the longest distance found so far.

4. Report each segment and the longest segment.

Flowchart

```
START
  |
Read ordered coordinate tuples
  |
Set longest distance = 0
  |
For each successive point pair, calculate distance
  |
Is distance greater than current longest?
  | yes -> Update longest segment
  |
Display segment distances and longest
  |
 END
```

Pseudocode

```
longest_distance = -1
FOR i from 0 to number_of_points - 2:
    p1 = points[i]; p2 = points[i + 1]
    distance = sqrt((x2-x1)^2 + (y2-y1)^2)
    DISPLAY segment and distance
    IF distance > longest_distance:
        update longest_distance and segment
DISPLAY longest segment
```

Python Code

```
from math import sqrt

points = [(0, 0), (3, 4), (9, 4), (10, 12)]
longest = (-1, None)

for i in range(len(points) - 1):
    x1, y1 = points[i]
    x2, y2 = points[i + 1]
    distance = sqrt((x2 - x1)**2 + (y2 - y1)**2)
    print(f"Segment {i + 1}: {distance:.2f}")
    if distance > longest[0]:
        longest = (distance, i + 1)

print(f"Longest segment: {longest[1]} ({longest[0]:.2f} units)")
```

Sample Output

```
Segment 1: 5.00
Segment 2: 6.00
Segment 3: 8.06
Longest segment: 3 (8.06 units)
```

Note: This example uses planar Euclidean distance. For latitude/longitude coordinates, use a geographic distance formula such as haversine.